

DSA -- Study Notes Index

Format: Concept + Example + 40s script + Follow-ups. PSU MSTC interview ready.

Revision Order (Follow This)

Step 1	01-DS-BASICS-AND-ARRAYS.md	Classification + address formula (numerical qs)
Step 2	02-LINKED-LIST-STACK-QUEUE.md	LIFO/FIFO + Infix/Prefix/Postfix (output qs)
Step 3	03-TREES-AND-GRAPHS.md	Traversals + BFS vs DFS (trace qs)
Step 4	04-HEAP-AND-HASHING.md	Max/Min Heap + Hashing + Collision resolution
Step 5	05-SORTING-AND-COMPLEXITY.md	Big O/omega/ + all 5 sorts + Master Theorem
Step 6	06-ADVANCED-ALGORITHMS.md	DP, Dijkstra, Kruskal, Huffman, NP-Complete

Day-before quick revision: Cheat sheets at bottom of each file only.

Files

File
Topics
Status

01-DS-BASICS-AND-ARRAYS.md

DS classification, 6 operations, Array address formula, 1D/2D, Linear vs Binary search

[OK] Done

02-LINKED-LIST-STACK-QUEUE.md

4 LL types, Stack (LIFO/PUSH/POP), Infix/Prefix/Postfix, Queue (FIFO), Priority Queue, Circular Queue

[OK] Done

03-TREES-AND-GRAPHS.md

Binary tree types, 3 traversals (Pre/In/Post), B-Tree, Graph, BFS vs DFS

[OK] Done

04-HEAP-AND-HASHING.md

Max/Min Heap, array index formula, Heapify, B-Tree vs B+Tree, Hashing, Collision resolution (Chaining vs Open Addressing)

[OK] Done

05-SORTING-AND-COMPLEXITY.md

Big O/Omega/Theta, Bubble/Selection/Insertion/Quick/Merge/Radix Sort, Recursion, Master Theorem

[OK] Done

06-ADVANCED-ALGORITHMS.md

Dynamic Programming, Dijkstra/Bellman-Ford, Kruskal/Prim MST, Huffman Coding, NP-Complete, Backtracking

[OK] Done

Master Cheat Sheet

DS Classification:

Primitive int, float, char, pointer
Linear Array, Linked List, Stack, Queue
Non-Linear Tree, Graph

Array:

Address = Base + i * size
Size = UB - LB + 1
Access = O(1) random
Linear Search O(n), works on unsorted
Binary Search O(log n), needs sorted

Linked List (4 types):

Singly one pointer, forward only
Doubly two pointers, bidirectional

Circular last first (no NULL)
 Circular Doubly doubly + circular
 Stack:
 LIFO Last In First Out
 PUSH TOP = TOP + 1
 POP TOP = TOP - 1
 Overflow = push on full | Underflow = pop on empty
 Notations:
 Infix A + B (operator between) normal
 Prefix +AB (operator before) Polish
 Postfix AB+ (operator after) Reverse Polish
 Queue:
 FIFO First In First Out
 Enqueue at REAR, Dequeue from FRONT
 Initial: F = R = -1
 Priority Queue higher priority dequeued first Heap preferred $O(\log n)$
 Circular Queue rear wraps to front no wasted space
 Binary Tree Types:
 Full every node: 0 or 2 children
 Complete all levels full except last (left right)
 Perfect all levels completely full
 Traversals:
 Preorder Root, L, R (NLR) root FIRST
 Inorder L, Root, R (LNR) root MIDDLE BST gives sorted!
 Postorder L, R, Root (LRN) root LAST
 B-Tree: self-balancing, $O(\log n)$ all ops, used in DB indexes, all leaves same level
 Graph: V + E, can have cycles
 BFS Queue (FIFO), level-by-level, shortest path in unweighted graph
 DFS Stack/recursion, deep-first, cycle detection, topological sort
 Heap:
 Complete Binary Tree with heap property
 Max Heap parent $\geq$ children root = max
 Min Heap parent $\leq$ children root = min
 Index formula (1-based): Left=2i, Right=2i+1, Parent= i/2
 Heapify= $O(n)$, Insert= $O(\log n)$, Delete= $O(\log n)$, Peek= $O(1)$
 B+Tree leaf nodes linked faster range queries used in MySQL
 Hashing:
 Key hash function index $O(1)$ access
 Division: $K \bmod n$ | Mid-Square | Folding
 Collision Chaining (linked list per bucket) OR Open Addressing
 Open Addressing: Linear Probing, Quadratic Probing, Double Hashing
 Sorting ($O(n)$):
 Bubble compare adjacent, swap stable, best $O(n^2)$
 Selection find min, move to front unstable, always $O(n^2)$
 Insertion insert into sorted part stable, best $O(n^2)$
 Sorting ($O(n \log n)$):
 Quick pivot + partition unstable, worst $O(n^2)$, best in practice
 Merge divide + merge stable, always $O(n \log n)$, needs $O(n)$ space
 Heap heapify + extract unstable, always $O(n \log n)$, $O(1)$ space
 Asymptotic Notations:
 Big O (O) upper bound worst case
 Omega (omega) lower bound best case
 Theta () tight bound average case
 Master Theorem: $T(n)=aT(n/b)+f(n)$
 Compare $f(n)$ with $n^{\log_b a}$
 $f(n) \leq n^{\log_b a}$ ($n^{\log_b a}$)
 $f(n) = n^{\log_b a}$ ($n^{\log_b a} \cdot \log n$)
 $f(n) \geq n^{\log_b a}$ ($f(n)$)

Time Complexity -- All Data Structures

[Target] This is a must-know table. Expect direct questions like "What is insertion time for BST?"

Data Structure
 Access
 Search
 Insert
 Delete
 Notes

Array

$O(1)$

$O(n)$ / $O(\log n)^*$

$O(n)$

$O(n)$

* $O(\log n)$ only if sorted + binary search

Singly Linked List

$O(n)$

$O(n)$

$O(1)$ at head

$O(1)$ at head

$O(n)$ at arbitrary position

Stack

$O(n)$

$O(n)$

$O(1)$ PUSH

$O(1)$ POP

Only top is directly accessible

Queue

$O(n)$

$O(n)$

$O(1)$ enqueue

$O(1)$ dequeue

Only front/rear accessible

BST (Average)

$O(\log n)$

$O(\log n)$

$O(\log n)$

$O(\log n)$

Balanced tree

BST (Worst)

$O(n)$

$O(n)$

$O(n)$

$O(n)$

Skewed/degenerate tree

B-Tree

$O(\log n)$

$O(\log n)$

$O(\log n)$

$O(\log n)$

Always balanced by design

Hash Table (Avg)

$O(1)$

$O(1)$

$O(1)$

$O(1)$

Assuming good hash function

Hash Table (Worst)

$O(n)$

$O(n)$

$O(n)$

$O(n)$

All keys collide

Heap (Min/Max)

$O(1)$ top
 $O(n)$
 $O(\log n)$
 $O(\log n)$
Top element always $O(1)$

Priority Queue (Heap)
 $O(1)$
 $O(n)$
 $O(\log n)$
 $O(\log n)$
Preferred over array PQ

B-Tree -- Properties Deep Dive

For a B-Tree of order m :

Property
Value

Max children per node
 m

Min children per node (non-root)
 $m/2$

Max keys per node
 $m - 1$

Min keys in root
1

Min keys in other nodes
 $m/2 - 1$

Internal nodes formula
 $m/2$

All leaves
Same level (always balanced)

Data maintained
Sorted

Example -- B-Tree of order 3 ($m=3$):
- Max children = 3
- Min children (non-root) = $3/2 = 2$
- Max keys = 2
- Min keys (non-root) = 1
Why B-Tree over BST for databases?

BST can become skewed $O(n)$ worst case. B-Tree is always balanced guaranteed $O(\log n)$. Also, B-Tree nodes hold multiple keys fewer disk reads (perfect for database indexes).

Sorting Algorithms -- Time Complexity (Quick Reference)

Algorithm
Best
Average
Worst
Space
Stable?

Bubble Sort
 $O(n^2)$
 $O(n^2)$
 $O(n^2)$
 $O(1)$
[OK]

Selection Sort
 $O(n^2)$
 $O(n^2)$
 $O(n^2)$
 $O(1)$

Insertion Sort
 $O(n^2)$
 $O(n^2)$
 $O(n^2)$
 $O(1)$
[OK]

Merge Sort
 $O(n \log n)$
 $O(n \log n)$
 $O(n \log n)$
 $O(n)$
[OK]

Quick Sort
 $O(n \log n)$
 $O(n \log n)$
 $O(n)$
 $O(\log n)$

Heap Sort
 $O(n \log n)$
 $O(n \log n)$
 $O(n \log n)$
 $O(1)$

Mnemonic for worst-case $O(n^2)$ sorts: "Bubble Selection Insertion = BSI = Bahut Slow hai India mein"
Mnemonic for $O(n \log n)$ sorts: "Merge Quick Heap = MQH = Meri Quick Ho gayi sorting!"

Topics To Add (Next Pages)

[x] Sorting algorithms -- Bubble, Selection, Insertion, Merge, Quick ([OK] in 05-SORTING-AND-COMPLEXITY.md)
[x] B-Tree properties + time complexity ([OK] in README + 03-TREES-AND-GRAPHS.md)
[x] Time complexity master table -- all DS ([OK] in README)
[x] Hashing -- hash function, collision handling ([OK] in 04-HEAP-AND-HASHING.md)
[x] Heap -- Max/Min, operations, array indexing ([OK] in 04-HEAP-AND-HASHING.md)
[x] Asymptotic Notations + Master Theorem ([OK] in 05-SORTING-AND-COMPLEXITY.md)

- [] Dynamic Programming -- concept + classic problems
- [] Graph algorithms -- Dijkstra, Prim, Kruskal (if asked in interview)